

Problema A – Espino obsesivo

Espino es un chico obsesivo con la exactitud. Cada vez que se va a reunir con sus amigos no puede resistir la necesidad de ser el encargado de repartir los pasteles o las pizzas para asegurar que todos y cada uno de los asistentes reciban una porción perfectamente equitativa. Él se ha asegurado de evaluar cada una de las pastelerías y pizzerías en las que pueden comprar y ya tiene sus proveedores predilectos, que son quienes tienen la mayor precisión simétrica en su fabricación. Espino siempre trae consigo una herramienta que funciona como una cinta métrica especial que le ayuda a saber exactamente la longitud de lado a lado de la pizza o pastel y que además le ayuda a encontrar el centro lo más exacto que puede, él siempre toma la medida antes de realizar la repartición, así como también marca el centro. Ahora quiere innovar en la mejora de su herramienta, añadiendo una pantalla digital y un dispositivo de entrada que le ayude a introducir la métrica tomada y la cantidad de personas y con ello la pantalla le muestre la longitud exacta que debe tener el borde curvo de la rebanada para entregar una porción perfecta desde la perspectiva de Espino, pues sus habilidades de corte también las ha ido mejorando hasta considerarse experto. Ayuda a Espino a implementar el algoritmo de solución que deberá integrar a su herramienta para calcular el tamaño de cada rebanada.

Entrada

Un número real L que representa la longitud de la pizza o pastel de extremo a extremo, seguido por un entero P que representa la cantidad de personas entre las que se hará la repartición.

Salida

Una línea con un número real T con 2 dígitos decimales (con redondeo) que indique la longitud del borde curvo de la rebanada.

Entrada de ejemplo 1	Salida de ejemplo 1
50.61 13	12.23

Notas

$$10 \leq L \leq 100$$

$$1 \leq P \leq 50$$

NOTA: Para este problema asuma que el valor de π es 3.1416

Problema B – Buen gobernante

El magnánimo doctor Víctor Von Doom a decidido crear un sistema de transporte en Lat-veria para que sus súbditos puedan viajar a través del país. Para lograr esto, decidió construir una serie de trenes entre ciudades, las líneas de tren son dirigidas, es decir que exista un tren que vaya de la ciudad A a la ciudad B no significa que hay un tren que va de la ciudad B a la ciudad A .

Para que sea funcional Doom se aseguró de que siempre existe un camino entre cualesquiera 2 ciudades. Además, para que todos puedan (tengan que) asombrarse con la grandeza de Doom, los trenes están contruidos de manera que cualquier viaje que empiece y termine en la misma ciudad, tiene que pasar por la capital.

Ahora has sido contratado obligatoriamente para evaluar la grandeza del sistema de trenes. Las ciudades están numeradas del 0 al $N - 1$, donde la capital es la ciudad 0. La grandeza del sistema es igual a la mínima cantidad de veces que uno tiene que pasar por la capital si empezando en la capital, se hace un viaje a la ciudad 1, después a la ciudad 2, después a la ciudad 3 y así consecutivamente hasta la ciudad $N - 1$.

Además, como las necesidades del país siempre cambian, se le harán Q modificaciones al sistema de trenes, cada modificación consiste en agregar o eliminar una línea de tren. Después de cada modificación debes responder la grandeza del sistema.

Entrada

En la primera línea 2 números N ($1 \leq N \leq 10^4$) y M ($N \leq M \leq \min(5 * 10^5, \frac{N(N-1)}{2})$), la cantidad de ciudades y la cantidad de líneas de tren que existen.

En las siguientes M líneas, 2 números a_i, b_i ($0 \leq a_i, b_i, \leq N - 1$) indicando que existe un tren que va de la ciudad a_i a la ciudad b_i

En la siguiente línea un número Q ($1 \leq Q \leq 10^3$), la cantidad de modificaciones al sistema de trenes que se harán. En las siguiente Q líneas, habrá 2 números a_j, b_j ($0 \leq a_j, b_j, \leq N - 1$) indicando que, si existía una línea de trenes entre a_j y b_j , esta será eliminada, y en el caso contrario será agregada.

Se te asegura que el sistema de trenes inicial, así como todos los que ocurren tras cada una de las operaciones cumple las condiciones dadas en la descripción del problema.

Salida

$Q + 1$ números, la grandeza inicial del sistema de trenes, y la grandeza después de aplicar cada una de las operaciones.

Entrada de ejemplo 1	Salida de ejemplo 1
5 5	1
0 1	1
1 2	1
2 3	2
3 4	
4 0	
3	
2 0	
0 3	
2 3	

Notas

Considera que las operaciones son permanentes, entonces cuando se aplica la segunda operación, la primer operación se mantiene.

Problema C – Replicando Jujutsu

Antonia es una estudiante de ingeniería de software que está siguiendo un tutorial sobre **Jujutsu (jj)**, un moderno sistema de control de versiones. Durante su lectura, descubre que **jj** le asigna a cada modificación en el código un identificador persistente llamado **Change ID**, el cual está representado por una cadena de exactamente 8 letras minúsculas.

Lo que más le fascinó a Antonia es la usabilidad en la terminal. Al ejecutar el comando **jj log**, el sistema no solo muestra los identificadores, sino que **resalta automáticamente el prefijo más corto que hace que cada Change ID sea único** en todo el repositorio. Así, los desarrolladores solo necesitan leer y escribir ese prefijo corto para referenciar su trabajo. Visualmente, la consola de Antonia muestra algo como esto:

```
@ qpvuntsm antonia@uni.edu 2026-09-17 "Update README"
```

```
|
o kkmpptxz antonia@uni.edu 2026-09-16 "Fix typo"
```

```
|
o qpvrstxy antonia@uni.edu 2026-09-15 "Initial commit"
```

Fascinada por esta funcionalidad, Antonia se ha propuesto un reto: escribir su propio algoritmo para replicar este cálculo. Dada una lista de **Change IDs** únicos, el programa debe calcular el prefijo mínimo único para cada uno de ellos, y luego imprimir la lista resultante ordenada alfabéticamente.

Entrada

La primera línea contiene un número entero N ($1 \leq N \leq 100\,000$), indicando la cantidad de **Change IDs** en el repositorio.

Las siguientes N líneas contienen una cadena de exactamente 8 letras minúsculas (de la 'a' a la 'z'), representando los **Change IDs** únicos.

Salida

Imprime N líneas. Cada línea debe contener el prefijo único más corto de uno de los identificadores. Esta lista de prefijos debe imprimirse **ordenada alfabéticamente**.

Entrada de ejemplo 1 <pre>3 qpvuntsm kkmpptxz qpvrstxy</pre>	Salida de ejemplo 1 <pre>k qpvr qpvu</pre>
Entrada de ejemplo 2 <pre>4 aaaaaaab abcabcab xyzxyzxy aaaaaaac</pre>	Salida de ejemplo 2 <pre>aaaaaaab aaaaaaac ab x</pre>

Notas

Explicación del ejemplo:

- Para **kkmpptxz**, ningún otro ID empieza con **k**, por lo que su prefijo mínimo único es **k**.
- Para **qpvuntsm** y **qpvrstxy**, ambos comparten el prefijo **qpv**. El cuarto carácter rompe el empate: **u** para el primero y **r** para el segundo. Por lo tanto, necesitan los prefijos **qpvu** y **qpvr** respectivamente para ser únicos.

$10^5 - 10^{10}$

10, 10, 10...

21

- Al ordenar la lista de prefijos obtenidos (k, qpvu, qpvr) alfabéticamente, la salida final es k, qpvr, qpvu.

Problema D – Decidiendo el concurso

Andres necesita ayuda para armar el concurso de la Coding Cup, y le a pedido ayuda a Sebastian para ello. Sebastian tiene N cajas. En la i -esima caja hay dos numeros a_i, b_i indicando que hay a_i problemas en esta caja, y que tienen dificultad b_i , un numero entero que va desde el 1 hasta k .

Un examen esta conformado por k problemas de cada una de las diferentes dificultades. Dadas las cajas de Sebastian, encuentra la cantidad de exámenes diferentes que Andres puede hacer.

Entrada

En la primera linea dos numeros $N(1 \leq N \leq 100000)$ y $K(1 \leq K \leq 100)$, los valores descritos en el problema.

En las siguientes N lineas, dos numeros a_i, b_i ($1 \leq a_i \leq 100, 1 \leq b_i \leq k$), los valores de la i -esima caja.

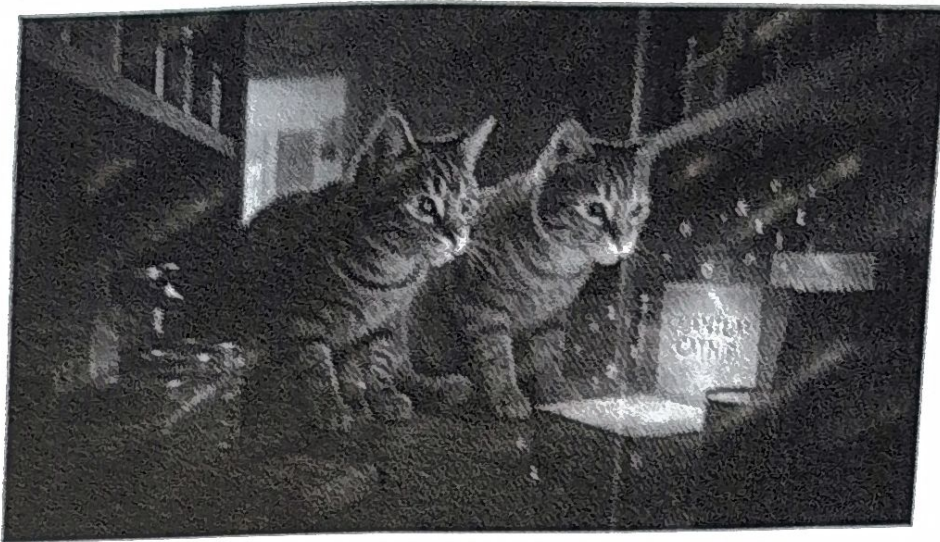
Salida

Solo un numero, la cantidad de exámenes diferentes que puede hacer Andres. Como este numero puede ser muy grande, imprimelo modulo $10^9 + 7$

Entrada de ejemplo 1 3 3 2 1 2 2 2 3	Salida de ejemplo 1 8
Entrada de ejemplo 2 4 3 1 2 1 1 1 1 1 2	Salida de ejemplo 2 0
Entrada de ejemplo 3 5 3 1 1 1 2 1 3 1 2 1 1	Salida de ejemplo 3 4

Problema E – Catnip Cuántico

Chatito y Cheetos han asaltado la alacena y han encontrado n cajas mágicas de catnip. La i -ésima caja tiene un nivel de potencia igual a a_i .



Dado que se trata de catnip cuántico del Michi-universo, el nivel de satisfacción total que obtiene un gato no es la suma de los niveles de potencia de las cajas que consumió, sino la **suma XOR** de dichos valores.

Los gatos deciden repartirse el botín jugando un juego con las siguientes reglas:

1. Juegan por turnos. Chatito siempre toma el primer turno.
2. En cada turno, un gato debe elegir exactamente una caja de catnip disponible, comérsela y retirarla del juego.
3. El juego termina cuando no quedan más cajas (después de exactamente n turnos).

El objetivo de cada gato es que su nivel de satisfacción XOR final sea **estrictamente mayor** que el del otro gato. Si ambos gatos terminan con exactamente el mismo nivel de satisfacción XOR, compartirán la caja de cartón para dormir y se declarará un **Empate**.

Asumiendo que tanto Chatito como Cheetos son genios matemáticos y juegan de manera óptima, determina quién ganará el juego o si terminará en empate.

Entrada

La primera línea contiene un único entero t ($1 \leq t \leq 10^4$) — el número de casos de prueba.

La primera línea de cada caso de prueba contiene un único entero n ($1 \leq n \leq 10^5$) — el número de cajas de catnip.

La segunda línea de cada caso de prueba contiene n enteros $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^9$) — los niveles de potencia de cada caja.

Se garantiza que la suma de n sobre todos los casos de prueba no excede 10^5 .

Salida

Para cada caso de prueba, imprime en una nueva línea el resultado del juego:

- Chatito si Chatito ganará el juego.
- Cheetos si Cheetos ganará el juego.
- Empate si ambos gatos terminarán con la misma satisfacción XOR.

11011
101001
11011

20 37 15 12
↑ ↑

3 3 3 3
↑ ↑

(Imprime las respuestas sin comillas. Presta atención a las mayúsculas).

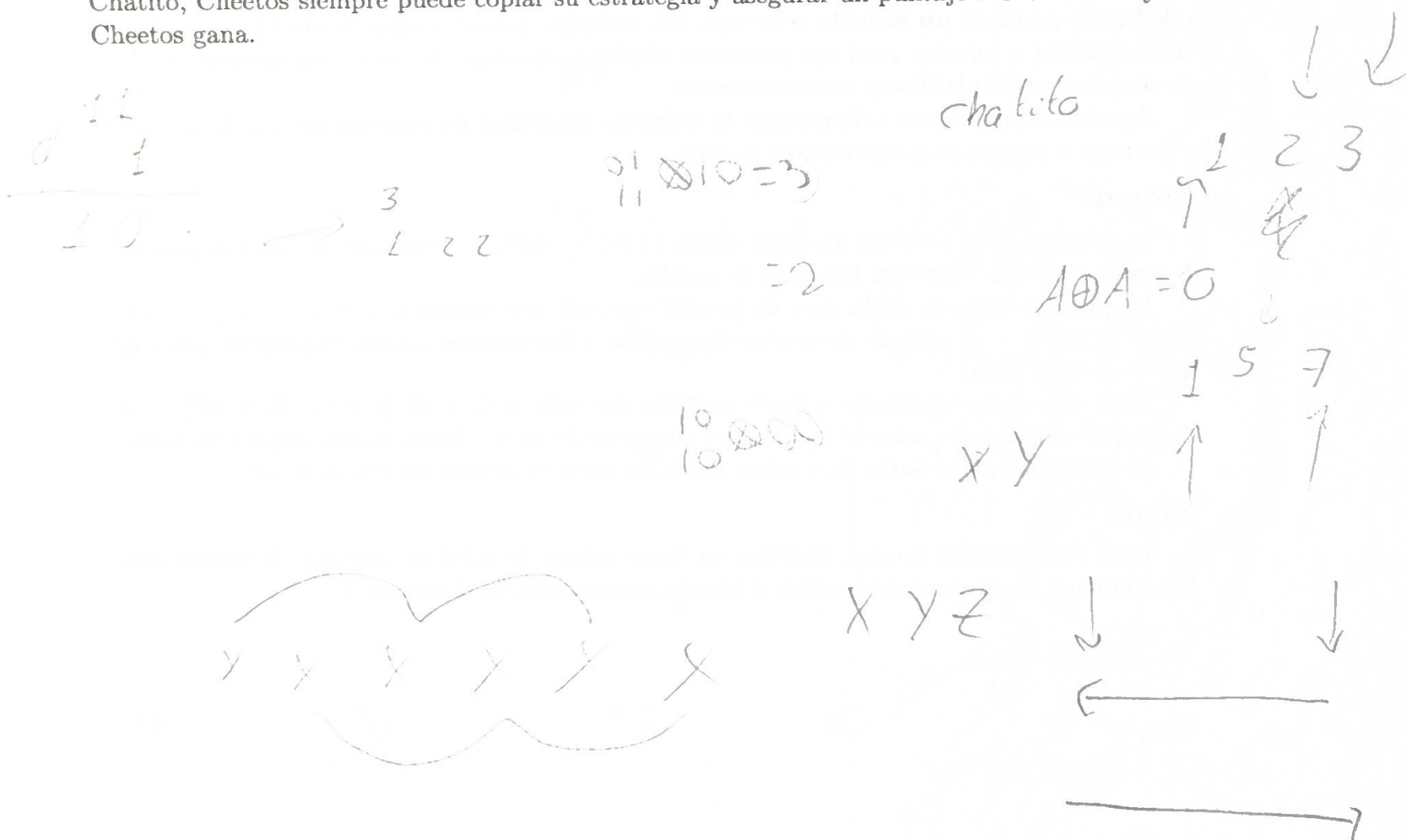
Entrada de ejemplo 1	Salida de ejemplo 1
3 3 1 2 2 4 3 3 3 3 3 3 3 3 3 0 3	Chatito Empate Cheetos

Notas

En el **primer caso de prueba**, Chatito puede elegir la caja con valor 1 en su primer turno. Quedan dos cajas con valor 2. Sin importar cuál elija Cheetos en su turno, Chatito tomará la última caja con valor 2. Al final, la satisfacción de Chatito será $1 \oplus 2 = 3$, y la de Cheetos será 2. Como $3 > 2$, Chatito gana.

En el **segundo caso de prueba**, el XOR total de todas las cajas es $3 \oplus 3 \oplus 3 \oplus 3 = 0$. Independientemente de cómo jueguen, la satisfacción de Chatito (X) y la de Cheetos (Y) siempre cumplirán la propiedad $X \oplus Y = 0$, lo que implica matemáticamente que $X = Y$. El juego siempre terminará en empate hagan lo que hagan.

En el **tercer caso de prueba**, el XOR total inicial es 3. Se puede demostrar matemáticamente que, al haber una cantidad impar de cajas, sin importar el primer movimiento de Chatito, Cheetos siempre puede copiar su estrategia y asegurar un puntaje XOR final mayor. Cheetos gana.



Problema F – El Gran Banquete de Cuauhtémoc

Cuauhtémoc está organizando una gran cena en su casa para todos sus amigos. Siendo un apasionado de la gastronomía, recientemente ha desarrollado una enorme fascinación por el uso del horno, por lo que para su cena ha impuesto una regla estricta: **todos los platillos que se sirvan en la cena deben ser horneados.**

Para maximizar la eficiencia, Cuauhtémoc rentará un horno exclusivo para cada receta que decida cocinar; es decir, dispone de una **cantidad infinita de hornos** que operan en paralelo de forma independiente.

Todos los platillos deben servirse en el mismo instante: exactamente dentro de T minutos a partir de ahora (en el minuto T). Cuauhtémoc tiene una lista de n recetas disponibles, numeradas del 1 al n . Para la i -ésima receta:

- Requiere C_i minutos de preparación manual en la mesa de la cocina.
- Una vez terminada su preparación, se introduce inmediatamente a su respectivo horno y toma H_i minutos en hornearse.
- Cuauhtémoc cuenta con campanas térmicas de última tecnología. Gracias a esto, **una vez que un platillo sale del horno, puede esperar fuera de él indefinidamente sin enfriarse ni perder calidad**, siempre y cuando haya terminado de hornearse en o antes del minuto T .

Además, como Cuauhtémoc está practicando, en esta ocasión va a estar solo en la cocina: solo puede preparar **un platillo a la vez**. Sin embargo, mientras algunos platillos se hornean en sus hornos o reposan bajo sus campanas térmicas esperando la cena, Cuauhtémoc es libre de preparar otros platillos o tomar descansos.

Ayuda a Cuauhtémoc a determinar la **máxima cantidad de recetas** que puede preparar y hornear a tiempo para servir las en la cena.

Entrada

La primera línea contiene un único entero t ($1 \leq t \leq 10^4$) — el número de casos de prueba. A continuación se describen los casos de prueba.

La primera línea de cada caso de prueba contiene dos enteros n y T ($1 \leq n \leq 2 \cdot 10^5$, $1 \leq T \leq 10^{14}$) — el número de recetas disponibles y los minutos totales disponibles antes de que se sirva la cena.

Cada una de las siguientes n líneas contiene dos enteros C_i y H_i ($1 \leq C_i, H_i \leq 10^9$) — el tiempo de preparación manual y el tiempo de horneado de la i -ésima receta, respectivamente.

Se garantiza que la suma de n sobre todos los casos de prueba no excede $2 \cdot 10^5$.

Salida

Para cada caso de prueba, imprime un único entero: la máxima cantidad de recetas que Cuauhtémoc puede preparar y servir a tiempo para la cena en el minuto T .

Entrada de ejemplo 1	Salida de ejemplo 1
3	3
4 100	1
20 70	2
15 50	
40 40	
10 30	
2 50	
30 30	
25 20	
3 40	
15 10	
15 10	
15 10	

20-70 15-50 40-40 10-30 20

↓

15-50 10-40 10-30

40

↓ ↓
3

15-15
↓ ↓
10 40
11 11
25 40

0001
0010
0011
0100
0101
0110
0111
1000
1001

0 20 30 45 85

20 70 10 30
10 30 40 40 20 70
15 50 15 50
40 40

Problema G – Atsa y los Combos

Atsa acaba de comprar el último Mortal Kombat. El juego cuenta con un conjunto de botones disponibles: *A, B, X, Y, L, R, UP, DOWN, LEFT* y *RIGHT*.

En el juego existen N combos, y cada combo consiste en una secuencia específica de botones que debe ser presionada en un orden determinado. Por ejemplo, un combo podría ser *DOWN, RIGHT, A*, mientras que otro podría ser *A, B, X*.

Atsa ha conseguido registrar una secuencia de M botones que fueron presionados durante una partida, pero no sabe exactamente qué combos fueron ejecutados.

Un combo pudo haber ocurrido si sus botones aparecen consecutivamente y en el orden indicado dentro de la secuencia registrada. Cada botón presionado puede formar parte de a lo más un combo, por lo que dos combos no pueden compartir ninguna de las posiciones de la secuencia.

Dada la lista de combos y la secuencia de botones registrados, determina la cantidad máxima de combos que pudieron haber sido ejecutados.

Entrada

- En la primera línea, un entero N ($1 \leq N \leq 1 \times 10^5$).
- Cada una de las siguientes N líneas contendrá una secuencia de botones por cada combo separados por espacios en blanco.
- La última línea contendrá la secuencia de M ($1 \leq M \leq 1 \times 10^6$) botones que Atsa registró.

Salida

Imprime un único entero: la cantidad máxima de combos que pudieron haber sido ejecutados.

Entrada de ejemplo 1 3 A B X B X X Y A B X Y	Salida de ejemplo 1 1
Entrada de ejemplo 2 5 UP LEFT A A B B X Y X Y UP LEFT UP LEFT A B X Y	Salida de ejemplo 2 3

Notas

- El total de botones presionados en todos los combos es a lo más 5×10^5 .

Problema H – Fichajes Deportivos

Al finalizar la Copa del Mundo, todos los grandes clubes de fútbol (incluyendo el tuyo) están evaluando a los mejores prospectos de la competencia para construir el mejor equipo a nivel de clubes del mundo. Después de un largo proceso de reclutamiento y visorías, tu agente de reclutamiento te entregó el reporte de los mejores N prospectos del mundial que aún no han sido comprados por otros clubes.

Pero, como quieres construir el mejor equipo posible, también necesitarás el mejor cuerpo técnico posible que pueda trabajar con estos N jugadores. Tu tarea es contratar el mejor equipo de entrenadores/ascesores para este selecto grupo. Según el reporte de tu agente, cada uno de los M entrenadores es compatible en habilidades y personalidad con un subconjunto de estos N jugadores.

Tu tarea es encontrar un subconjunto de preparadores con el que cada uno de los N jugadores tenga al menos un preparador con el que es compatible y que el costo de contratación sea el mínimo posible.

Entrada

En la primera línea de entrada, dos enteros $M(1 \leq M \leq 1000)$ y $N(1 \leq N \leq 15)$.

En la segunda línea de entrada, M enteros; el i -ésimo entero $C_i(1 \leq C_i \leq 10^9)$ representa el costo de contratación del i -ésimo preparador.

Seguido de M líneas, cada una de estas M líneas tendrá:

$K_i(1 \leq K_i \leq 15)$, el número de jugadores con los que el i -ésimo preparador es compatible, seguido de K_i enteros, cada uno representando el número de jugador con el que es compatible (los jugadores están numerados del 1 al N).

Salida

Un único entero representando el *menor costo de contratación* de preparadores, que cubra con todos los N jugadores.

En caso de no poder cubrir a los N jugadores de ninguna manera, deberás imprimir -1 .

Entrada de ejemplo 1	Salida de ejemplo 1
4 6 9 6 6 7 5 1 2 3 4 5 3 1 2 3 3 4 5 6 1 6	12

Problema I – CtrlAltTec

El área de Tecnologías de la Información del Tecnológico Nacional de México (TecNM) detectó un gusano informático al que ya bautizaron como **CtrlAltTec**. Entró por el instituto con más computadoras conectadas a la red (ahí tiene más puertas por las cuales colarse), y solo se puede propagar de un instituto a otro a través de los M enlaces de fibra óptica que conectan directamente algunos pares de institutos. La misión del virus es reprobar a todos los estudiantes que tienen una calificación mayor a 80, y no queremos eso, ¿o sí?

La prioridad es proteger al instituto con menos computadoras porque es el que tiene menos personal de sistemas, y el que más tardaría en recuperarse si lo alcanza el gusano. Para evitarlo, puedes poner en cuarentena de red a algunos institutos intermedios (desconectarlos temporalmente de la fibra mientras se revisa una por una todas sus computadoras). Poner en cuarentena al instituto i cuesta exactamente c_i : su número de computadoras, que es justo lo que hay que revisar antes de reconectarlo.

Nunca hace falta, ni tiene caso, poner en cuarentena al instituto donde empezó el brote (el gusano ya salió de ahí) ni al instituto que estás protegiendo (con la falta de personal, no pueden hacerlo a tiempo). Tu trabajo es encontrar el conjunto de institutos intermedios más barato, en computadoras totales, que corte cualquier ruta posible de contagio entre esos dos institutos. Si están conectados por fibra directamente entre sí, no hay absolutamente nada que puedas hacer.

Entrada

La primera línea contiene dos enteros N y M : el número de institutos del TecNM (numerados del 1 al N) y el número de enlaces de fibra óptica.

La segunda línea contiene N enteros $c_1, c_2, \dots, c_N$: el número de computadoras conectadas a la red en cada instituto. Todos los c_i son distintos entre sí, así que el instituto con más computadoras (donde entró el gusano) y el de menos (el que hay que proteger) siempre están bien definidos.

Cada una de las siguientes M líneas contiene dos enteros u y v : hay un enlace de fibra óptica directo entre los institutos u y v . No hay dos enlaces entre el mismo par de institutos.

Salida

Si es imposible evitar que el gusano llegue al instituto protegido (porque está conectado por fibra directamente al instituto donde empezó el brote), imprime un solo entero: -1 .

En cualquier otro caso, imprime un solo entero: el costo mínimo total (en computadoras) para poner en cuarentena institutos intermedios y así cortar cualquier ruta entre los dos institutos.

Entrada de ejemplo 1	Salida de ejemplo 1
5 6 50 10 30 5 80 1 2 2 3 3 4 1 3 3 5 2 5	30

Entrada de ejemplo 2	Salida de ejemplo 2
2 1 7 20 1 2	-1

Notas

- $2 \leq N \leq 1000$
- $0 \leq M \leq 3000$
- $1 \leq c_i \leq 1,000,000$, y todos los c_i son distintos entre sí
- $1 \leq u, v \leq N$, $u \neq v$, para cada enlace de fibra óptica
- No hay dos enlaces entre el mismo par de institutos

Problema J – Código de Calidad

En una fábrica textil de Uriangato, cada rollo de tela recibe un código formado únicamente por letras mayúsculas y dígitos.

Un código se considera de buena calidad si cumple todas las siguientes condiciones:

- Contiene al menos una letra A mayúscula.
- Contiene al menos un dígito.
- La cantidad de letras es mayor que la cantidad de dígitos.

Dado el código de un rollo de tela, determina si es de buena calidad.

Entrada

La entrada contiene una línea con una cadena S formada únicamente por letras mayúsculas y dígitos. La cadena S contiene entre 3 y 100 caracteres.

Salida

En la primera línea imprime CALIDAD si el código cumple todas las condiciones. En caso contrario, imprime NO CALIDAD.

En la segunda línea imprime la cantidad de letras y la cantidad de dígitos utilizando el siguiente formato:

Letras: X

Dígitos: Y

donde X es la cantidad de letras y Y es la cantidad de dígitos que contiene el código.

Entrada de ejemplo 1 TELA2026A	Salida de ejemplo 1 CALIDAD Letras: 5 Dígitos: 4
Entrada de ejemplo 2 A123456	Salida de ejemplo 2 NO CALIDAD Letras: 1 Dígitos: 6

Problema K – Calculando el Frenado

Un sistema de asistencia al frenado de un automóvil utiliza una regla sencilla para estimar qué tan fuerte debe ser el frenado.

El sistema recibe dos valores enteros:

- *P*: presión ejercida sobre el pedal de freno, entre 0 y 100.
- *D*: distancia al obstáculo, entre 0 y 100 metros.

Para representar el grado de pertenencia al conjunto *frenado fuerte*, el sistema combina ambos parámetros mediante la siguiente fórmula:

$$F = \frac{2P + (100 - D)}{3}$$

Donde *F* es un valor entero entre 0 y 100. Debes asegurarte que al hacer la división, utilices una división entera (lo que significa que, al dividir dos números, se redondea el resultado hacia el entero menor más cercano).

La interpretación de la fórmula es:

- Una mayor presión sobre el pedal aumenta la necesidad de frenado.
- Una menor distancia al obstáculo aumenta la necesidad de frenado.
- El valor *F* representa el grado de pertenencia al conjunto *frenado fuerte*, expresado como un valor entero de 0 a 100.

Se considera que el sistema requiere un **frenado fuerte** cuando $F \geq 70$. En caso contrario, se considera un **frenado moderado**.

Tu tarea es calcular *F* y mostrar el tipo de frenado correspondiente.

Entrada

La entrada contiene una sola línea con dos números enteros *P* y *D*:

$$0 \leq P \leq 100$$

$$0 \leq D \leq 100$$

Salida

Imprime en una sola línea el texto correspondiente al tipo de frenado, seguido de un espacio y el valor entero *F*.

Si $F \geq 70$, imprime:

FRENADO FUERTE *F*

En caso contrario, imprime:

FRENADO MODERADO *F*

Entrada de ejemplo 1 80 20	Salida de ejemplo 1 FRENADO FUERTE 80
Entrada de ejemplo 2 100 100	Salida de ejemplo 2 FRENADO MODERADO 66
Entrada de ejemplo 3 0 0	Salida de ejemplo 3 FRENADO MODERADO 33
Entrada de ejemplo 4 65 20	Salida de ejemplo 4 FRENADO FUERTE 70

Problema L – Numeración Dorada

En la deslumbrante ciudad de Asjabad, capital de Turkmenistán, un exclusivo consorcio de magnates tecnológicos construyó el residencial más excéntrico del mundo: “Las Mansiones de Oro”. Para destacar la opulencia del lugar, los arquitectos mandaron fundir la numeración de cada residencia en placas de oro puro de 24 quilates. Inspirados en la elegancia de las civilizaciones antiguas como la maya, decidieron no usar el sistema decimal convencional, sino un sistema numérico posicional de base 20 (vigesimal). Para la representación visual de los 20 dígitos necesarios, combinaron las cifras decimales con las primeras letras del alfabeto inglés:

$$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, G, H, I, J\}$$

Donde los dígitos del 0 al 9 conservan su valor original, y las letras de la A a la J representan los valores del 10 al 19 respectivamente ($A = 10, B = 11, \dots, I = 18, J = 19$).

Dado el altísimo costo de fundir cada símbolo en oro puro, la constructora estipuló que en esta primera etapa del proyecto las casas solo utilizarían como máximo 2 dígitos en base 20 (lo que abarca valores decimales del 0 al 399, como 10 o JJ). Sin embargo, anticipando futuras expansiones del residencial a miles de lotes adicionales, exigieron que el software de gestión sea capaz de escalar hasta 8 dígitos en base 20.

Un lunes por la mañana, el sistema automatizado de recepción de paquetería del condominio colapsó por un corte de energía. En ese instante llegó el mensajero de “Mantomandados Express”, un joven estudiante de primer semestre de Ingeniería en Sistemas Computacionales con la mochila llena de paquetes y el tiempo en contra. Al querer iniciar las entregas, el chico, desconcertado, descubrió que los paquetes tenían escritas las direcciones en formato decimal, pero al buscar la casa correspondiente se topaba con los lingotes de oro grabados con códigos como 1A o 1J. El estudiante recordó rápidamente sus clases de Matemáticas Discretas y las fórmulas para realizar las conversiones necesarias:

Método de Potencias (Base 20 a Decimal): Dada una cadena de k dígitos $d_{k-1}d_{k-2} \dots d_1d_0$ en base 20, su valor equivalente en sistema decimal N_{10} se calcula mediante la suma ponderada de potencias de 20:

$$N_{10} = \sum_{i=0}^{k-1} d_i \cdot 20^i = d_{k-1} \cdot 20^{k-1} + d_{k-2} \cdot 20^{k-2} + \dots + d_1 \cdot 20^1 + d_0 \cdot 20^0$$

Método de Divisiones Sucesivas (Decimal a Base 20): Dado un número entero en decimal N_{10} , se divide de forma sucesiva entre la base 20 obteniendo cocientes q_j y residuos r_j :

$$\begin{aligned} N_{10} &= q_0 \cdot 20 + r_0, & 0 \leq r_0 < 20 \\ q_0 &= q_1 \cdot 20 + r_1, & 0 \leq r_1 < 20 \\ q_1 &= q_2 \cdot 20 + r_2, & 0 \leq r_2 < 20 \\ &\vdots \\ q_{m-1} &= 0 \cdot 20 + r_m, & 0 \leq r_m < 20 \end{aligned}$$

El número resultante en base 20 se obtiene concatenando los residuos mapeados a sus respectivos símbolos en orden inverso a su obtención: $r_m r_{m-1} \dots r_1 r_0$.

Como el chico lleva su laptop en la mochila, decidió escribir un programa que lea las órdenes de entrega y convierta automáticamente las direcciones, soportando tanto las casas actuales de 2 dígitos como la expansión futura de hasta 7 dígitos. ¡Ayúdalo a implementar el algoritmo!

Entrada

La primera línea de la entrada estándar contiene un entero n ($1 \leq n \leq 10^4$), que representa la cantidad de conversiones que se deben procesar. Las siguientes n líneas contienen una consulta cada una con el formato:

[tipo] [número] Donde:

- [tipo] es un carácter D o M.
- D: Especifica que el [número] proporcionado está en sistema Decimal.
- M: Especifica que el [número] proporcionado está en sistema Maya/Vigesimal (Base 20).
- [número] es la representación numérica a convertir.

Salida

Para cada consulta de entrada, imprime una única línea en la salida estándar con ambos formatos separados por un espacio en el siguiente orden: [numero_en_decimal] [numero_en_base_20]

Entrada de ejemplo 1	Salida de ejemplo 1
7	20 10
D 20	20 10
M 10	415 10F
D 415	419 10J
M 10J	399 JJ
M JJ	400 100
D 400	7999 JJJ
M JJJ	

2432 00000000
 127 99999999
 7594999999
 1111111111

$$20_{20} = 2 \times 20 +$$

$$M 10 \rightarrow 1 \times 20 + 0 \times 20 = 20$$

$$20_{10} = 111_{20}$$

48 - 1 26

$$20 \sqrt{20}$$

JAC

J JJJ JJJJ

"20"
 ↑